

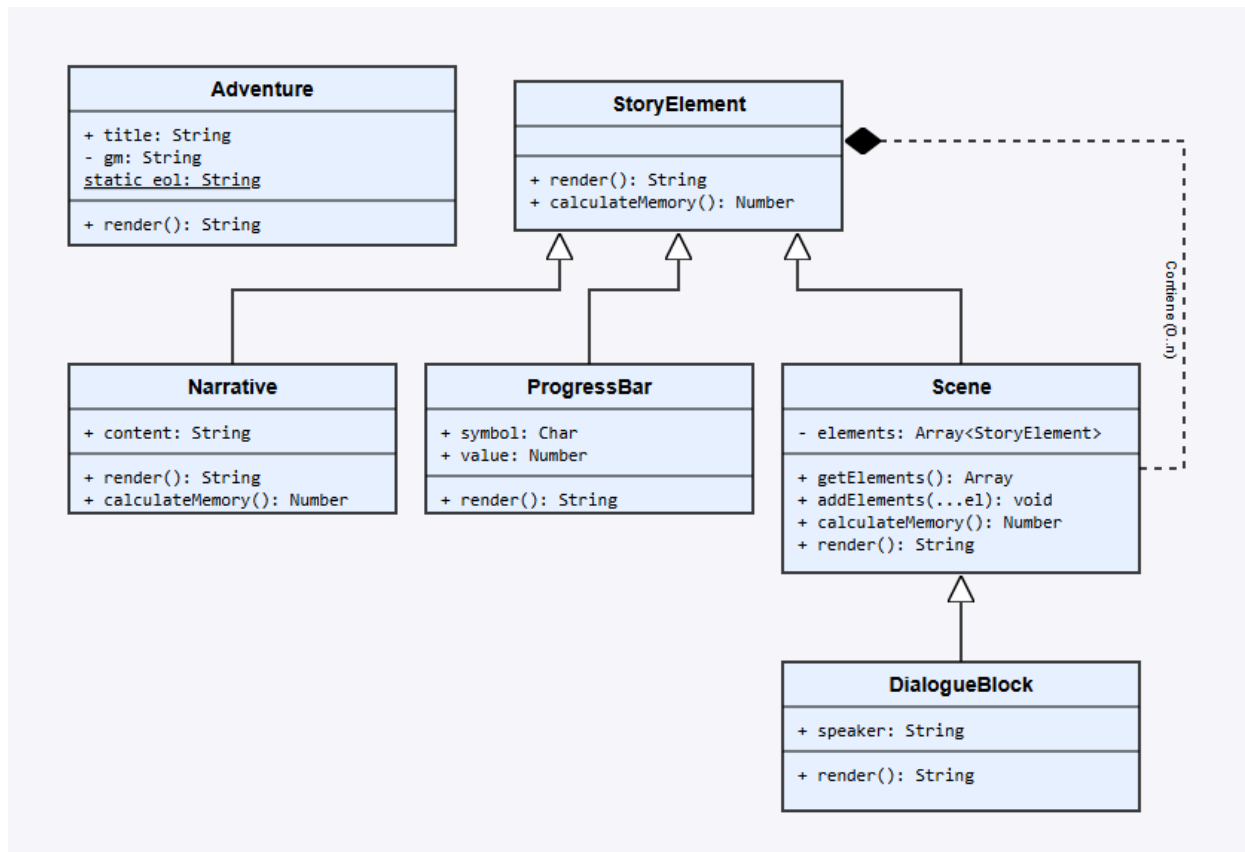
Ejercicio: Programación Orientada a Objetos

Motor de Aventuras de Texto (StoryEngine)

Contexto

Se desea implementar el núcleo de un motor de videojuegos para aventuras conversacionales (aventuras de texto). El sistema debe permitir organizar la narrativa en diferentes elementos y calcular el "coste de memoria" (basado en palabras) de la aventura.

Diagrama de Clases



Estructura de Clases y Requisitos

1. StoryElement (*01.story-element.test.js*)

- 1.1. Clase **StoryElement**: Crear la clase.
- 1.2. Método **render()**: No tiene parámetros. Debe devolver una cadena vacía.
- 1.3. Método **calculateMemory()**: No tiene parámetros. Devuelve el coste de memoria (número) del elemento. En los elementos base sin contenido devuelve 0.

2. Adventure (*02.adventure.test.js*)

- 2.1. Clase **Adventure**: Crear la clase.
- 2.2. Propiedades:
 - **Título (title)**: Se establece al crear la instancia (constructor).
 - **Game Master (gm)**: Es una propiedad **oculta**. Si no se proporciona en el constructor, debe asignarse el valor "IA".
- 2.3. Propiedad calculada **gm**: Debe devolver el valor de la propiedad oculta.
- 2.4. Modificar **gm**: Se debe poder modificar el Game Master como una propiedad normal.
 - *Validación*: El nombre debe tener **más de 3 caracteres** y no puede tener espacios sobrantes delante o detrás (**trim**). Si no cumple esto, el valor no cambia.
- 2.5. Propiedad estática **eol (End of Line)**: Propiedad de clase (no de instancia).
 - Por defecto debe ser "**\n**".
 - Se debe poder modificar (ej. cambiar a "**
**").

- **2.6. Método `render()`:** Debe devolver una cadena con la cabecera de la aventura:
 - "Aventura: [Título]"
 - "Master: [GM]"
 - "Fecha: [creationDate]"
 - (Cada línea separada por el valor de `eol`).

3. Narrative (*03.narrative.test.js*)

- **3.1. Clase `Narrative`:** Hereda de `StoryElement`.
- **3.2. Constructor:** Recibe además un texto (`content`) como parámetro. Si no se pasa nada, se almacena una cadena vacía.
- **3.3. Método `render()`:** Devuelve el contenido del texto seguido del salto de línea de la aventura (`Adventure.eol`).
- **3.4. Método `calculateMemory()`:** Cuenta las palabras del contenido (separadas por espacios). Varios espacios seguidos no cuentan como palabras extra.

4. ProgressBar (*04.progress-bar.test.js*)

- **4.1. Clase `ProgressBar`:** Hereda de `StoryElement`.
- **4.2. Constructor:** Recibe dos parámetros: `symbol` (carácter) y `value` (longitud).
 - Si no se pasan parámetros, por defecto el símbolo es `#` y el valor es 20.
- **4.3. Método `render()`:** Devuelve una cadena con el `symbol` repetido tantas veces como indique `value`, seguido de un salto de línea de `Adventure`.

5. Scene (*05.scene.test.js*)

- **5.1. Clase `Scene`:** Hereda de `StoryElement`. Representa una escena o capítulo que contiene otros elementos.

- **5.2. Componentes:**
 - Tiene un campo oculto para almacenar los elementos hijos (`elements`).
 - Tiene un método `getElements()` que devuelve el array.
 - Al crearse, está vacío.
- **5.3. Método `addElements(...elements)`:** Permite añadir varios elementos a la vez.
- **5.4. Restricción de tipo:** Solo añade elementos si son instancias de `StoryElement` (o herederos). Si se intenta añadir un número, fecha u otro objeto, lo ignora. Permite añadir otras `Scene` dentro (anidamiento).
- **5.5. Método `calculateMemory()`:**
 - Si la escena está vacía, devuelve 0.
 - Si tiene elementos, devuelve la **suma** del `calculateMemory()` de todos sus hijos (recursividad).
- **5.6. Método `render()`:** Concatena el resultado del método `render()` de todos sus hijos.

6. DialogueBlock (*06.dialogue-block.test.js*)

- **6.1. Clase `DialogueBlock`:** Hereda de `Scene`.
- **6.2. Restricción de contenido:** Es un contenedor especial que **solo acepta** elementos de tipo `Narrative`. Si se intenta añadir una `ProgressBar` o una `Scene`, no se inserta.
- **6.3. Propiedad `speaker`:**
 - Se pasa como parámetro al constructor (ej. "Heroe").
 - Si no se pasa, se queda como cadena vacía.
- **6.4. Método `render()` sin `speaker`:**
 - Si `speaker` está vacío, imprime todos los textos igual que la clase `Scene`.

- Ejemplo (con `Adventure.eol='\n'` y dos elementos `Narrative` con “*Hola Mundo*” y “*Qué tal estás*” respectivamente):

`Hola mundo\nQué tal estás\n`

- **6.5. Método `render()` con `speaker`:**

- Si `speaker` tiene valor, imprime cada línea de diálogo precedida por el nombre y dos puntos.
- Ejemplo (igual que el ejemplo anterior pero con `speaker='Heroe'`):

`Heroe: Hola mundo\nHeroe: Qué tal estás\n`

- **6.6. Compatibilidad:** Debe respetar el `Adventure.eol` si este cambia dinámicamente.

Instrucciones de Archivos y Módulos

Para que los tests automáticos funcionen correctamente, debes seguir estrictamente estas convenciones:

1. Nomenclatura de Archivos

El nombre del archivo debe ser la versión en **minúsculas y separada por guiones** (kebab-case) del nombre de la clase.

Nombre de la Clase (Código)	Nombre del Archivo (.js)
MiClase	mi-clase.js
SuperHeroe	super-hero.js

2. Exportación e Importación

Utiliza el sistema **CommonJS** estándar de Node.js.

Forma correcta de EXPORTAR (al final de tu archivo):

```
class MiClaseGenerica {  
  // ... tu código ...  
}  
module.exports = MiClaseGenerica;
```

Forma correcta de IMPORTAR (si una clase necesita a otra):

```
// Si necesitas usar MiClaseGenerica dentro de OtraClase
const MiClaseGenerica = require('./mi-clase-generica');

class OtraClase extends MiClaseGenerica {
  // ...
}

module.exports = OtraClase;
```

Instrucciones de Entrega

Comprime y sube toda la carpeta menos la carpeta **node_modules**.